FORMULARIOS PHP

¿Qué es un formulario?

Un **formulario** es una parte clave en las aplicaciones web. Es la única forma que tienen los usuarios de enviar información a través de una página web, como nombres, correos electrónicos, contraseñas, etc. Los formularios suelen estar construidosn HTML y los datos que el usuario introduce se envían al servidor para ser procesados, en este caso, con **PHP**.

Partes de un Formulario

Un formulario HTML básico tiene varios elementos clave:

- **Campos de entrada** (`input`) donde el usuario introduce datos.
- **Método de envío** (`GET` o `POST` por ahora) que determina cómo se envían los datos al servidor.
- **Acción** (`action`), que define a qué archivo de PHP se enviarán los datos.

Ejemplo de Formulario HTML:

```
<form action="procesar_formulario.php" method="POST">
     Nombre: <input type="text" name="nombre"><br>
     Correo electrónico: <input type="email" name="email"><br>
     <input type="submit" value="Enviar">
</form>
```

En este caso:

- El **action** indica que los datos se enviarán al archivo, en este caso al archivo `procesar_formulario.php.` Si no se especifica nada, se envía al mismo archivo o php que lo llamó.´
- El **method** está configurado como `POST` , lo que significa que los datos se enviarán de manera segura y no aparecerán en la barra de direcciones del navegador.
- Los campos de entrada tienen un **atributo name** (`name="nombre"` y `name="email"`), que es cómo PHP podrá acceder a esos datos en el servidor.

Campos de entrada

Tabla: Tipos de `<input>` en HTML5

Tipo (`type`)	Propósito / Uso	Validación automática del navegador	Notas útiles
`text`	Texto general (nombres, títulos...)	No (solo si `required`)	Acepta cualquier cadena
`password`	Contraseñas (oculta caracteres)	No (solo si `required`)	Acepta texto, lo oculta visualmente
`email`	Dirección de correo electrónico	Formato de email	Permite múltiples con `multiple`
`number`	Números	Solo números (rango opcional)	`min`, `max`, `step`
`tel`	Teléfono	No valida formato	Puedes usar `pattern`
`url`	Dirección web	Debe empezar por `http://` o `https://`	Útil para links
`search`	Búsqueda (como `text`, pero semántico)	No	Estilizado por navegadores
`date`	Selección de fecha	Solo fechas válidas	Formato controlado por navegador
`time`	Solo hora (`hh:mm`)		Sin fecha, solo hora
`datetime-local`	Fecha y hora		No incluye zona horaria
`month`	Mes y año (`YYYY-MM`)		Útil para resúmenes mensuales
`week`	Semana del año (`YYYY-W##`)		Poco común, pero estándar
`checkbox`	Verdadero/falso (selección múltiple)	No (usa `required` si al menos uno debe marcarse)	Enviar varios con el mismo `name[]`
`radio`	Selección exclusiva entre opciones	Si se agrupan con el mismo `name`	Solo uno puede estar seleccionado

Tipo (`type`)	Propósito / Uso	Validación automática del navegador	Notas útiles
`file`	Subir archivos	Solo si `required`	Usa `accept="image/*"` para filtrar
`hidden`	Campo no visible	No	Útil para datos internos
`range`	Barra deslizante	Solo valores numéricos	Usa `min`, `max`, `step`
`color`	Selector de color	Hexadecimal (`#RRGGBB`)	Soporte variable entre navegadores
`submit`	Enviar formulario	—	Puede tener un `value` como texto
`reset`	Restaurar valores del formulario	—	No envía datos, solo resetea
`button`	Botón genérico	—	Usa `onclick` o JS

- **Todos** estos campos pueden tener atributos como `required`, `placeholder`, `pattern`, `min`, `max`, etc.
- Para enviar datos, el campo debe tener un atributo `name`, si no, **no se incluye en `$_GET` o `$_POST`**.
- Muchos navegadores modernos aplican validación HTML5 automáticamente, **sin necesidad de JavaScript**.
- Si ponemos algún campo `"disabled"` este campo no se enviará en el formulario, aunque tenga valor, no llegará en `$_GET['var']` ni en `$_POST['var']`. Si usamos `readonly` sí se envía, el campo será no editable, pero si se enviará.

Vamos a ver algunos que tienen un poco más de dificultad:

```
  <p>Intereses (selecciona los que apliquen):</p>
    <label><input type="checkbox" name="intereses[]" value="tecnologia">
Tecnología</label><br>
    <label><input type="checkbox" name="intereses[]" value="deporte"> Deporte</label>
<br>
    <label><input type="checkbox" name="intereses[]" value="lectura"> Lectura</label>
<br>
    <label><input type="checkbox" name="intereses[]" value="viajes"> Viajes</label>
<br><br>

    <p>Género:</p>
    <label><input type="radio" name="genero" value="hombre"> Hombre</label><br>
    <label><input type="radio" name="genero" value="mujer"> Mujer</label><br>
    <label><input type="radio" name="genero" value="otro"> Otro</label><br><br>

    <label>
      País:
      <select name="pais">
        <option value="es">España</option>
        <option value="mx">México</option>
        <option value="ar">Argentina</option>
      </select>
    </label><br><br>
```

Para ver cómo recibimos los datos podemos impirmir `$_GET` o `$_POST` para ver qué estamos recibiendo.

Métodos de Envío: GET vs POST

- **GET**: Los datos se envían como parte de la URL (visible en la barra de direcciones) y es útil cuando no hay información sensible involucrada. Es más limitado en cuanto a la cantidad de datos que se pueden enviar. Se utiliza principalmente para solicitudes que no modifican datos en el servidor, como búsquedas o filtros

  Ejemplo de URL con datos en **GET**:

  ```
  http://miweb.com/procesar_formulario.php?
  nombre=Juan&email=juan@example.com`
  ```

  - Los espacios son borrados y reemplazados con el carácter +
  - Algunos caracteres no alfanuméricos (vocales con tilde, ñ, etc) se sustituyen con los valores hexadecimales.

    - La página y la información codificada están separados por el «?» carácter.
    - Los parámetros se separan por el operador &
    - Tiene un limite a enviar, hasta sólo 1.000 caracteres (puede configurarse en el servidor Web)
    - GET no puede ser utilizado para enviar datos binarios, como imágenes o documentos de texto al servidor.
- **POST**: Los datos se envían en el cuerpo de la solicitud HTTP, no son visibles en la URL y no tienen límites en cuanto a tamaño. Se utiliza para enviar datos sensibles o para realizar cambios en el servidor, como enviar un formulario de registro.
    - El método POST no tiene ninguna restricción en el tamaño de los datos a enviar.
    - El método POST se puede utilizar para enviar ASCII, así como datos binarios
    - El tipo de datos en el cuerpo se especifica en la cabecera Content-Type
    - Soporta datos de entrada ***multi-part binary*** a la hora se subir archivos al servidor
- `$_REQUEST` es una de las superglobales en PHP, que permite acceder a los datos enviados a través de formularios, ya sea mediante el método **GET**, **POST**, o *COOKIE*. Es una forma conveniente de obtener información de diferentes tipos de solicitudes HTTP. Es un array que contiene todas las variables que recibimos, lo cual nos simplifica a la hora de extraer cada elemento.

  Características de `$_REQUEST`:

  1. **Acceso a Datos**:
     - Contiene la información de los datos enviados por el usuario, que puede provenir de:
       - Parámetros de la URL (método GET)
       - Datos de formularios (método POST)
       - Cookies
  2. **Jerarquía de Preferencias**:
     - Cuando accedes a `$_REQUEST`, PHP busca el valor en el siguiente orden:
       1. **POST**: Si el dato se envía mediante POST, se obtendrá de `$_POST`.
       2. **GET**: Si no está en POST, se buscará en `$_GET`.
       3. **COOKIE**: Si no se encuentra en GET, se buscará en las cookies (`$_COOKIE`).

3. **Uso Común**:
    - Es útil cuando no estás seguro de qué método se usará para enviar el formulario, o cuando deseas manejar todos los datos de entrada en un solo array.
    - Sin embargo, es importante tener en cuenta que puede hacer que la validación sea más complicada, ya que no puedes distinguir fácilmente de dónde proviene cada dato.
4. **Ejemplo de Uso**:

```
// Suponiendo que el formulario ha sido enviado
    // Acceder a datos usando $_REQUEST
    $nombre = $_REQUEST['nombre'];
    $email = $_REQUEST['email'];
    // Procesar los datos
```

5. **Ventajas y Desventajas**:

- **Ventajas**:
    - Comodidad: Te permite acceder a los datos independientemente del método utilizado.
- **Desventajas**:
    - Ambigüedad: Puede ser menos claro de dónde provienen los datos, lo que puede complicar la validación y la depuración.
    - No se recomienda su uso si quieres asegurarte de la fuente de los datos, ya que usar `$_GET`, `$_POST`, o `$_COOKIE` directamente es más seguro y claro.

Procesando los Datos en PHP

`$_GET` y `$_POST` son variables superglobales que forman arrays asociativos de *keys* y *values*, donde los *keys* son los nombres del formulario (atributo "*name*") y los *values* son los datos de entrada de los usuarios. El hecho de que sean ***superglobals*** hace que sean accesibles desde el script independientemente del ámbito.

La validación de los datos es crucial para asegurar que el formulario ha sido rellenado correctamente y que la información recibida es válida y segura. En PHP, utilizamos `isset()` y `empty()` para validar la existencia y el contenido de los campos, junto con funciones adicionales como `filter_var()` para la validación específica de algunos tipos de datos, como correos electrónicos.

Pasos para validar los datos del formulario:

1. **Comprobar si los datos fueron enviados**: Para empezar, hay que asegurarse de que el formulario fue enviado. Esto se puede hacer verificando el método de envío con `$_SERVER['REQUEST_METHOD']`.

   ```php
   if ($_SERVER["REQUEST_METHOD"] == "POST") {
       // Aquí se procesan los datos del formulario
   }
   ```

2. **Validación de campos individuales**: Comprobar si los campos existen y no están vacíos, usamos `isset()` para verificar que los campos están definidos y no son nulos (han sido enviados) y `empty()` para asegurarnos de que contienen un valor significativo.

   `isset($var)` devuelve **`true` si la variable está definida y no es `null`**.

   `empty($var)` devuelve **`true` si la variable está vacía**, es decir, si su valor es considerado "falsy" en PHP:

   - `""` (cadena vacía)
   - `0` (entero)
   - `"0"` (cadena con cero)
   - `0.0`
   - `false`
   - `array()` (array vacío)
   - `null`
   - **y también si la variable no está definida**

   Diferencias clave

Valor de `$var`	`isset($var)`	`empty($var)`
`null`	false	true
`0`	true	true
`"0"`	true	true
`""` (cadena vacía)	true	true
`false`	true	true
`[]` (array vacío)	true	true
No definida	false	true

Supongamos que esperamos una URL como esta:

```
pagina.php?id=123&accion=editar
```

Queremos validar que:

1. `id` está presente, es numérico y no vacío.
2. `accion` existe y no está vacía.

```php
if (isset($_GET['id']) && is_numeric($_GET['id']) && $_GET['id'] > 0) {
 $id = (int) $_GET['id'];
} else {
 // Error: ID no válido
}

if (isset($_GET['accion']) && $_GET['accion'] !== '') {
 $accion = $_GET['accion'];
} else {
 // Error: acción no especificada
}
```

¿Podríamos usar empty?

```php
if (!empty($_GET['id'])) {
 // cuidado: esto da false si id = 0
}
```

Este caso **fallaría si** `id=0`, porque `empty()` lo considera vacío. Si `0` fuera un valor válido, usar `empty()` sería incorrecto.

En resumen, debemos usar `empty()` cuando nos da igual si te mandan `"0"`, `false`, `null`, etc. y solo quieres saber "¿hay algo aquí?". Usaremos `isset() && !== ''` si queremos más control sobre qué valores se aceptan.

Vamos a ver cómo validar el campo email suponiendo que también es obligatorio, pero además debe cumplir con el formato de un correo electrónico válido. Usamos la función `filter_var()` para validar su formato.

```
if (isset($_POST['email']) && !empty($_POST['email'])
            && filter_var($_POST['email'], FILTER_VALIDATE_EMAIL))
{     $email = htmlspecialchars($_POST['email']);
} else {
     echo "El campo correo es obligatorio y debe ser un email válido.<br>";
}
```

Aquí se asegura que el campo no está vacío y tiene el formato adecuado de un correo electrónico. Si no es así, se muestra un mensaje de error. En este caso funcionaría si quitamos el isset, pero es buena práctica indicar claramente que queremos comprobar existencia y contenido.

Una vez que todos los campos han sido validados, podemos procesar los datos, almacenarlos en una base de datos, o hacer cualquier otra operación necesaria.

Buenas Prácticas al Procesar Formularios

1. **Validación en el servidor**: Aunque puedes realizar validaciones en el lado del cliente con JavaScript o con HTML5, **nunca confíes solo en ello**. La validación en el servidor (con PHP) es imprescindible, ya que un usuario podría desactivar JavaScript o enviar datos maliciosos.

2. **Sanitización de datos**: Siempre debes limpiar los datos que recibes del usuario, ya que podrían incluir código malicioso. Usa funciones como `htmlspecialchars()` y `filter_var()` para evitar vulnerabilidades como **inyección de código** o **XSS** (Cross-Site Scripting).

   `htmlspecialchars()` **convierte caracteres especiales en entidades HTML**, para que el navegador **los muestre como texto plano** en lugar de interpretarlos como código HTML o JavaScript. Se usa para evitar inyecciones de HTML o JavaScript (XSS) cuando vas a **mostrar datos del usuario** en una página web. Ejemplo típico de ataque:

```
$_POST['nombre'] = '<script>alert("hack");</script>';
echo $_POST['nombre']; // ¡Ejecuta el script!`
```

Ahora con `htmlspecialchars()`:

```
echo htmlspecialchars($_POST['nombre']);
// Muestra: <script>alert("hack");</script>
```

El navegador **ya no ejecuta** el código, solo lo muestra como texto.

3. **Uso de POST para datos sensibles**: Para formularios donde el usuario envía información privada (como contraseñas o direcciones), usa siempre el método **POST**. **GET** deja los datos expuestos en la URL, lo que no es seguro.